Open in Colab

Python Tuple Objects

A *tuple* in Python is just an ordered collection of some Python objects.

To create a tuple, write some Python objects seperating them with a: `,`

In [1]:
```
'hello', 'world', 1, None, 1
```

Out[1]:
```
('hello', 'world', 1, None, 1)
```

Warning: When representing a tuple object, like in the previous example, Python always adds a pair of paranthesis around the tuple.

Writing the tuples in this way is a good practice.

Python also allows one more `,` at the end of the tuple. This extra `,` does not have an effect. The tuple created will be the same.

In [2]:
```
'hello', 'world', 1, None, 1,
```

Out[2]:
```
('hello', 'world', 1, None, 1)
```

So if you want to create a tuple with single object then just write the object and put a `,` at the end.

In [3]:
```
'hello',
```

Out[3]:
```
('hello',)
```

Warning: Writing `('hello')` without `,` does not create a tuple!

If you want to create an empy tuple use:

In [4]:
```
()
```

Out[4]:
```
()
```

or

In [5]:
```
tuple()
```

Out[5]:
```
()
```

Since a tuple is one Python object then you can assign it to a variable.

In [6]:
```
h = 'hello',
w = 'world', 1
```

Converting Strings To Tuples

In [7]:
```
tuple('hello')
```

```
Out[7]: ('h', 'e', 'l', 'l', 'o')
```

```
In [8]: tuple(('hello',))
```

```
Out[8]: ('hello',)
```

Then you can use `.join()` method to convert a tuple to a string.

```
In [9]: "".join(('h', 'e', 'l', 'l', 'o'))
```

```
Out[9]: 'hello'
```

```
In [10]: ":".join('hello')
```

```
Out[10]: 'h:e:l:l:o'
```

Operations on Tuples

Like strings:

You can get the length of a tuple by the builtin `len()` function.

```
In [11]: len(w)
```

```
Out[11]: 2
```

To test if an item is in the tuple use `in` keyword.

```
In [12]: 'world' in w, 'hello' in w
```

```
Out[12]: (True, False)
```

To test if an item is **not** in the tuple use `not in` keyword.

```
In [13]: 'world' not in w, 'hello' not in w
```

```
Out[13]: (False, True)
```

To count the number of occurences of an item in a tuple use `.count()` method.

```
In [14]: w.count('world'), w.count('hello')
```

```
Out[14]: (1, 0)
```

Python tuples can be concatenated by `+` operation.

```
In [15]: h + w
```

```
Out[15]: ('hello', 'world', 1)
```

Also you can use `*` to repeat the same tuple multiple times.

```
In [16]: (h + w) * 3
```

```
Out[16]: ('hello', 'world', 1, 'hello', 'world', 1, 'hello', 'world', 1)
```

Comparing Tuples

You can compare two tuples with `<`, `<=`, `>`, `>=`, `==` and `!=` operations.

In [17]:
```
h == ('hello',)
```

Out[17]: True

In [18]:
```
h + w > h
```

Out[18]: True

Here the comparision is done in *lexicographic* order.

Precedence of the `,` Operator

Warning: Precedence of the `,` operator is less than the other operators except the `=` operator. Therefore, when doing tuple operations, always use paranthesis around the `,` separated tuple items.

For example:

In [19]:
```
1, 2 + 3, 4
```

Out[19]: (1, 5, 4)

is equivalent to:

In [20]:
```
1, (2 + 3), 4
```

Out[20]: (1, 5, 4)

but not equivalent to:

In [21]:
```
(1, 2) + (3, 4)
```

Out[21]: (1, 2, 3, 4)

Nested Tuples

Paranthesis is used to create a tuple with another tuple:

In [22]:
```
a = 'one', 2, (3, (4, 5))
len(a)
```

Out[22]: 3

Here the items of the tuple are: `'one'`, `2` and `(3, (4, 5))`.

Accessing Tuple Items

To access the first item in a tuple use `[0]`, to access the second `[1]`, and so on.

In [23]:
```
a[0]
```

Out[23]: 'one'

To access the last item of a tuple use `[-1]`. You can use negative numbers to index the items

from end.

In [24]:
```
a[-1]
```

Out[24]:
```
(3, (4, 5))
```

To index a tuple from the beginning count like `0,1,2,...` To index a tuple from the end count like `-1,-2,-3,...`

To get the (positive) index of an item in a tuple use `.index()` method.

In [25]:
```
a.index((3,(4,5)))
```

Out[25]:
```
2
```

To access an item in a nested tuple use multiple `[...]`'s.

In [26]:
```
a[-1][1]
```

Out[26]:
```
(4, 5)
```

In [27]:
```
a[-1][1][0]
```

Out[27]:
```
4
```

Since we use the same notation to access the letters of a string, be careful when working on tuples of strings.

In [28]:
```
a[0][0]
```

Out[28]:
```
'o'
```

Here `a[0][0]` is the first character of the first item in the tuple.

Range of Indexes

You can get a part of a tuple by `[start:end]` notation.

In [29]:
```
a[0:2]
```

Out[29]:
```
('one', 2)
```

Here `a[0:2]` gets a tuple with `a[0]` and `a[1]` but not `a[2]`. You can also use negative indexes here.

In [30]:
```
a[0:-1]
```

Out[30]:
```
('one', 2)
```

If you ommit the start index Python assumes it `0`. So to get the first `2` items:

In [31]:
```
a[:2]
```

Out[31]:
```
('one', 2)
```

If you ommit the end index Python assumes it the length of the tuple. So to get the last `2` items:

In [32]:
```
a[-2:]
```

```
Out[32]: (2, (3, (4, 5)))
```

If your indexes are out of valid ranges then Python assumes them like empty:

```
In [33]: a[-10:10]
Out[33]: ('one', 2, (3, (4, 5)))
```

You can skip some items with `[start:end:skip]` notation. For example to get only the items in odd positions:

```
In [34]: a[::2]
Out[34]: ('one', (3, (4, 5)))
```

Unpacking Tuple Items

If you need you can assign different variables to each item of a tuple. This is called unpacking.

```
In [35]: a
Out[35]: ('one', 2, (3, (4, 5)))

In [36]: first, second, third = a

In [37]: first
Out[37]: 'one'

In [38]: second
Out[38]: 2

In [39]: third
Out[39]: (3, (4, 5))
```

Also you can do this nested if the tuple is nested.

```
In [40]: first, second, (first_of_third, (first_of_second_of_third, second_of_second_of_third))

In [41]: second_of_second_of_third
Out[41]: 5
```

Repacking Tuple Items

You can use `*` to unpack a tuple and use `,` to pack to a tuple. Here `*` prevents Python from creating a nested tuple instead of a flat one.

```
In [42]: *a, None
Out[42]: ('one', 2, (3, (4, 5)), None)
```

Compare this with:

```
In [43]: a, None
```

Out[43]: (('one', 2, (3, (4, 5))), None)